

講義 21：建立 Z 資料表與 Open SQL 寫入（授課順序：接在講義 15 之後）

對應練習：ex21 | 答案物件：資料表 ZTR21_STUD + 程式 ZR_TR21_ZTABLE

本講重點

- DDIC 三層件：Domain → Data Element → 表格欄位，各管什麼
- SE11 建立透明表：鍵欄位、Delivery Class、Technical Settings
- SM30 維護畫面（Table Maintenance Generator）
- Open SQL 寫入：INSERT / UPDATE / MODIFY / DELETE
- LUW 與 COMMIT WORK / ROLLBACK WORK 觀念

1. DDIC 三層件：欄位是怎麼組成的

講義 6 學過「讀」標準表；客製開發常需要自己的表存資料（參數表、log 表、暫存表——本專案的 ZDQM 系列就有）。SE11 的欄位定義是三層疊起來的：

層	管什麼	例
Domain（值域）	技術屬性：型別、長度、小數位、允許值清單（Value Range）、轉換常式	INT4、值域 0 ~ 999
Data Element（資料元素）	語意：欄位標籤（F1 說明、畫面上的欄位名稱，可多語言）	「學生成績」
表格欄位	引用一個 Data Element（或直接用內建型別）	SCORE TYPE ztr21_score

為什麼分三層？重複利用與一致性：十張表都有「成績」欄位時，共用同一個 Data Element，標籤、F1 說明、值域全系統一致，改一處全生效。標準表的欄位全是這樣組的——這也是為什麼 TYPE scarr-carrid 能帶出那麼多語意（講義 6）。

實務折衷：鍵欄位與有業務意義的欄位用 Data Element；純技術性欄位（旗標、備註）可直接用內建型別（CHAR、INT4...）省事。課程練習兩種都做。

2. SE11 建立透明表

步驟（細節照練習 ex21 的規格做）：

1. SE11 → Database table → 輸入 ZTR21_STUD → Create
2. Delivery Class：A（應用資料，最常用）。速查：

Class	用途
A	應用資料（主檔/交易資料）——預設選它
C	客戶自訂設定檔（Customizing，會跟著 TR 搬）

Class 用途

L 暫存資料

1. Data Browser/Table View Maint. : 選 **Display/Maintenance Allowed** (之後 SE16N/SM30 才能維護)
2. **Fields** 頁籤 : 第一欄一定是 **MANDT TYPE mandt** 且勾 **Key** (client 隔離 , 講義 0/6) ; 接著鍵欄位、資料欄位
3. **Technical Settings** (必填才能啟用) : Data Class **APPL0** (主檔類) 、Size Category **0** (預估筆數級距 , 練習表選最小)
4. Enhancement Category (選單 Extras) : 選 **Can be enhanced** 或 **Cannot be enhanced** 皆可 (練習表無所謂 , 正式表依團隊規範)
5. 啟用 (Ctrl+F3) ——表就真的建在資料庫了 , SE16N 可立刻查

命名 : 客製表 **Z** 開頭 (本課 **ZTR21_STUD**) ; 欄位名限 16 字元。表建錯欄位型別 , 上線後要改很痛 (要做轉檔) , 設計階段多想一分鐘。

3. SM30 維護畫面

讓使用者 (或顧問) 不寫程式就能維護表內容 :

1. SE11 該表 → Utilities → **Table Maintenance Generator**
2. Authorization Group 練習用 **&NC&** (不檢核) ; Function Group 給 **ZFG_TR21** (畫面程式的容器 , 講義 15 學過) ; Maintenance type 選 one step (單畫面)
3. 產生後 , SM30 輸入表名 → Maintain , 就有現成的新增/修改/刪除畫面

實務上參數表、對照表幾乎都配 SM30 ; 正式環境的維護權限與是否產 TR 由 Delivery Class 與權限控制。

4. Open SQL 寫入

讀是 SELECT ; 寫有四個指令 , 全部用 **sy-subrc** 回報結果 (鐵律不變) , **sy-dbcnt** 是影響筆數 :

```
DATA gs_stud TYPE ztr21_stud. " 直接用表名當結構型別
```

```
* INSERT : 新增一筆 ; 主鍵已存在 → sy-subrc = 4 , 不會蓋掉
```

```
gs_stud-id      = 'S0001'.
```

```
gs_stud-name    = '王小明'.
```

```
gs_stud-score   = 85.
```

```
INSERT ztr21_stud FROM gs_stud.
```

```
* UPDATE : 改既有資料 ; 找不到 → sy-subrc = 4
```

```
UPDATE ztr21_stud SET score = 90 WHERE id = 'S0001'.
```

```
* MODIFY : 有就改、沒有就新增 ( upsert ) ——參數表最愛用
```

```
MODIFY ztr21_stud FROM gs_stud.
```

```
* DELETE : 刪除
```

```
DELETE FROM ztr21_stud WHERE id = 'S0001'.
```

多筆版本：`INSERT ztr21_stud FROM TABLE gt_stud.` (`UPDATE/MODIFY/DELETE` 同理有 `FROM TABLE`) 。
注意 `INSERT FROM TABLE` 遇到任一筆主鍵重複就整批 **dump**，除非加 `ACCEPTING DUPLICATE KEYS` (重複的跳過、`sy-subrc = 4`) 。

- `MANDT` 欄位一樣不用 (不要) 自己塞，系統自動帶當前 `client` 。
- 寫入欄位建議程式帶齊稽核欄 (如異動者 `sy-uname`、異動日 `sy-datum`) ——查問題時會感謝自己。

5. LUW 與 COMMIT WORK

資料庫的變更不是逐句永久生效，而是以 **LUW (Logical Unit of Work)** 為單位，最後一次「確認」才真正落地：

```
INSERT ztr21_stud FROM gs_stud.
IF sy-subrc <> 0.
    ROLLBACK WORK.           " 整包撤銷：這個 LUW 內所有寫入取消
    MESSAGE '寫入失敗，已回復' TYPE 'E'.
ENDIF.
COMMIT WORK.                " 整包確認：全部永久生效
```

- **概念**：一個業務動作的多筆寫入 (如表頭 + 明細) 必須「全成功或全失敗」，不能寫一半——這就是 LUW 的意義。
- 報表程式跑完 (或畫面切換) 時系統會隱含 **commit**——所以練習程式沒寫 `COMMIT WORK` 資料多半也進去了，但正式程式的寫入要明確 **COMMIT / ROLLBACK**，把「哪裡算一個完整動作」寫清楚。
- 多人同時改同一筆怎麼辦？正式做法要配 **Lock Object** (`SE11` 建 `EZ...`，產生 `ENQUEUE/DEQUEUE FM`) ——本課先認識名詞，實作屬進階課題。

6. 常見錯誤與陷阱

症狀	原因
表啟用不了	Technical Settings 沒填 (<code>Data Class / Size Category</code>)
SE16N 看不到剛寫的資料	寫入時 <code>sy-subrc</code> 其實是 4 (沒檢查) ；或在別的 <code>client</code> 查
INSERT 一直 <code>subrc = 4</code>	主鍵重複——練習程式重跑前先 <code>DELETE</code> 舊測試資料，或改用 <code>MODIFY</code>
INSERT FROM TABLE 直接 dump	批次裡有主鍵重複，沒加 <code>ACCEPTING DUPLICATE KEYS</code>
SM30 說表不能維護	建表時 <code>Data Browser/Table View Maint.</code> 選了不允許，或沒產 <code>Maintenance</code> 畫面
自己塞 <code>MANDT</code>	不用，系統自動處理 (跟 <code>SELECT</code> 一樣)

7. 課堂練習

完成 `ex21`：建 Domain + Data Element + 透明表 `ZTR21_STUD`、產 `SM30` 維護畫面，再寫程式跑完 `INSERT / UPDATE / MODIFY / DELETE` 全流程並驗證 `sy-subrc` 。